

Jurnal Praktikum
Sistem Operasi
Modul 11: Memori Xinu

Nama : M Faris

Nim : 2311104017

Tujuan:

1. Mahasiswa mampu memahami struktur memori pada Xinu.

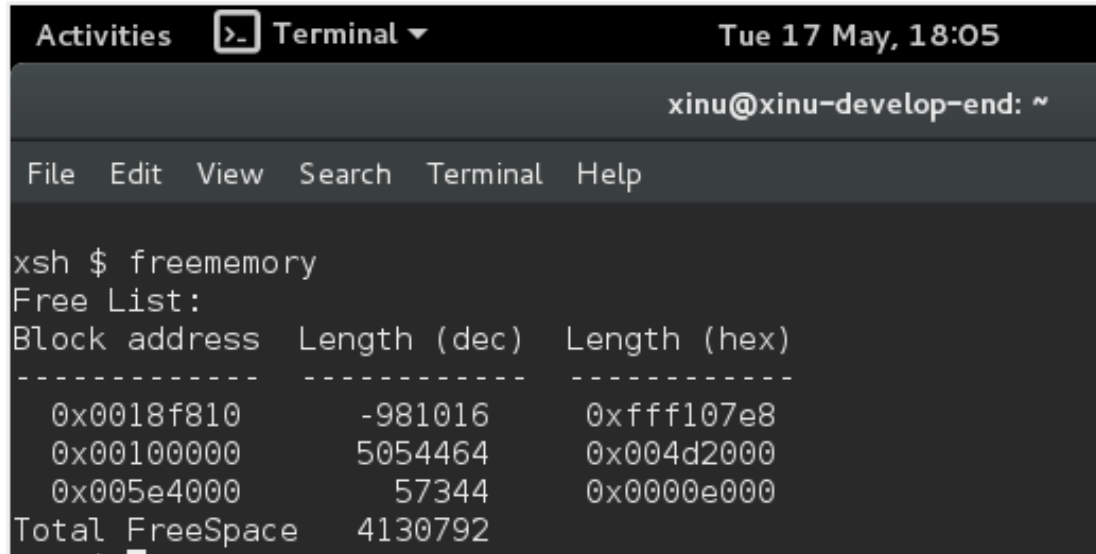
Catatan:

1. Praktikan wajib untuk screenshot setiap langkah yang dikerjakan hingga tampilan output akhir
2. Untuk soal source code, kumpulkan SS-nya saja
3. Praktikan wajib untuk melakukan screenshot lengkap dengan nama root. Contoh: root@username
4. Berikan identitas nama - nim dalam bentuk comment di Source Code
5. Harap kerjakan secara mandiri, jika tidak paham silahkan bertanya kepada Asisten Praktikum masing-masing. Dilarang mengcopy jawaban dan source code dari teman!

Jurnal

1. [80 poin] Buatlah perintah baru bernama **freememory** yang memiliki dua fungsi berikut:
 - a. [40 poin] Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - b. [40 poin] Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

Output yang diinginkan:

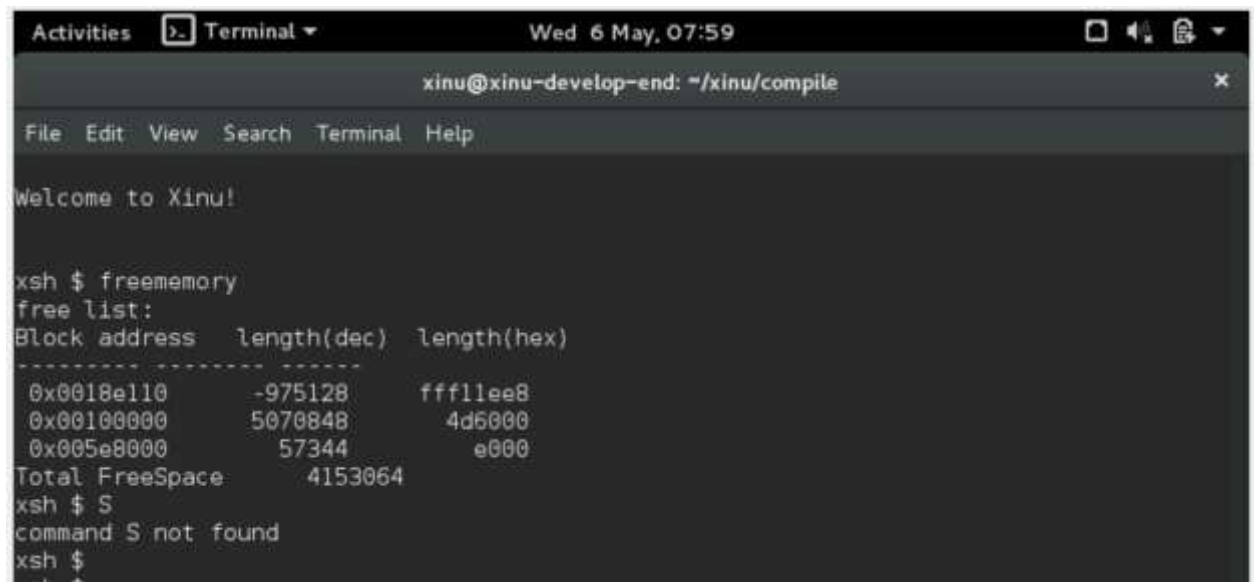


```
Activities Terminal Tue 17 May, 18:05
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help

xsh $ freememory
Free List:
Block address  Length (dec)  Length (hex)
-----
0x0018f810    -981016    0xffff107e8
0x00100000     5054464    0x004d2000
0x005e4000      57344      0x0000e000
Total FreeSpace 4130792
```

Jawab

Output:



```
Activities Terminal Wed 6 May, 07:59
xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help

Welcome to Xinu!

xsh $ freememory
free list:
Block address  length(dec)  length(hex)
-----
0x0018e110    -975128     fff11ee8
0x00100000     5070848     4d6000
0x005e8000      57344       e000
Total FreeSpace 4153064
xsh $ S
command S not found
xsh $
```

2. [4 poin per subsoal] Jawablah pertanyaan berikut:
- Mengapa Xinu memisahkan data segment dan BSS segment?
 - Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?
 - Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?
 - Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?
 - Apa tantangan utama dalam penggunaan heap di Xinu?

Jawab

- Pemisahan ini bertujuan untuk efisiensi penyimpanan dan manajemen memori. Data segment digunakan untuk variabel global yang sudah diinisialisasi (memerlukan ruang di file biner), sedangkan BSS (Block Started by Symbol) digunakan untuk variabel global yang belum diinisialisasi. BSS hanya perlu mencatat ukuran yang dibutuhkan dan akan di-nol-kan saat booting, sehingga memperkecil ukuran file executable
- Alokasi (`getmem`): Mengambil blok dari `memlist`, sehingga mengurangi ukuran total free space dan berpotensi memecah satu blok besar menjadi dua (fragmentasi).

Dealokasi (`freemem`): Mengembalikan blok ke `memlist`, meningkatkan ukuran free space, dan idealnya menggabungkan blok yang berdekatan (coalescing) untuk mengurangi fragmentasi

- Stack dikelola secara otomatis oleh sistem (LIFO) dan memiliki batas yang jelas per proses, sehingga risiko kebocoran kecil. Sebaliknya, Heap dikelola secara manual oleh programmer. Jika lupa membebaskan memori (memory leak) atau terjadi fragmentasi eksternal yang parah, sistem bisa kehabisan memori meskipun secara total masih ada ruang yang tersisa
- Karena jumlah dan ukuran blok memori bebas bersifat dinamis dan tidak menentu. Linked list memungkinkan penyisipan dan penghapusan blok di posisi mana pun secara efisien tanpa perlu menggeser elemen lain (seperti pada array), serta memudahkan pencarian blok yang ukurannya sesuai (misal menggunakan strategi First Fit).
- Tantangan utamanya adalah fragmentasi eksternal. Seiring waktu, memori bebas akan terpecah menjadi blok-blok kecil yang tersebar. Hal ini bisa menyebabkan kegagalan alokasi memori besar meskipun total free space mencukupi, karena tidak ada satu blok tunggal yang cukup besar dan kontigu (berurutan) untuk memenuhi permintaan tersebut